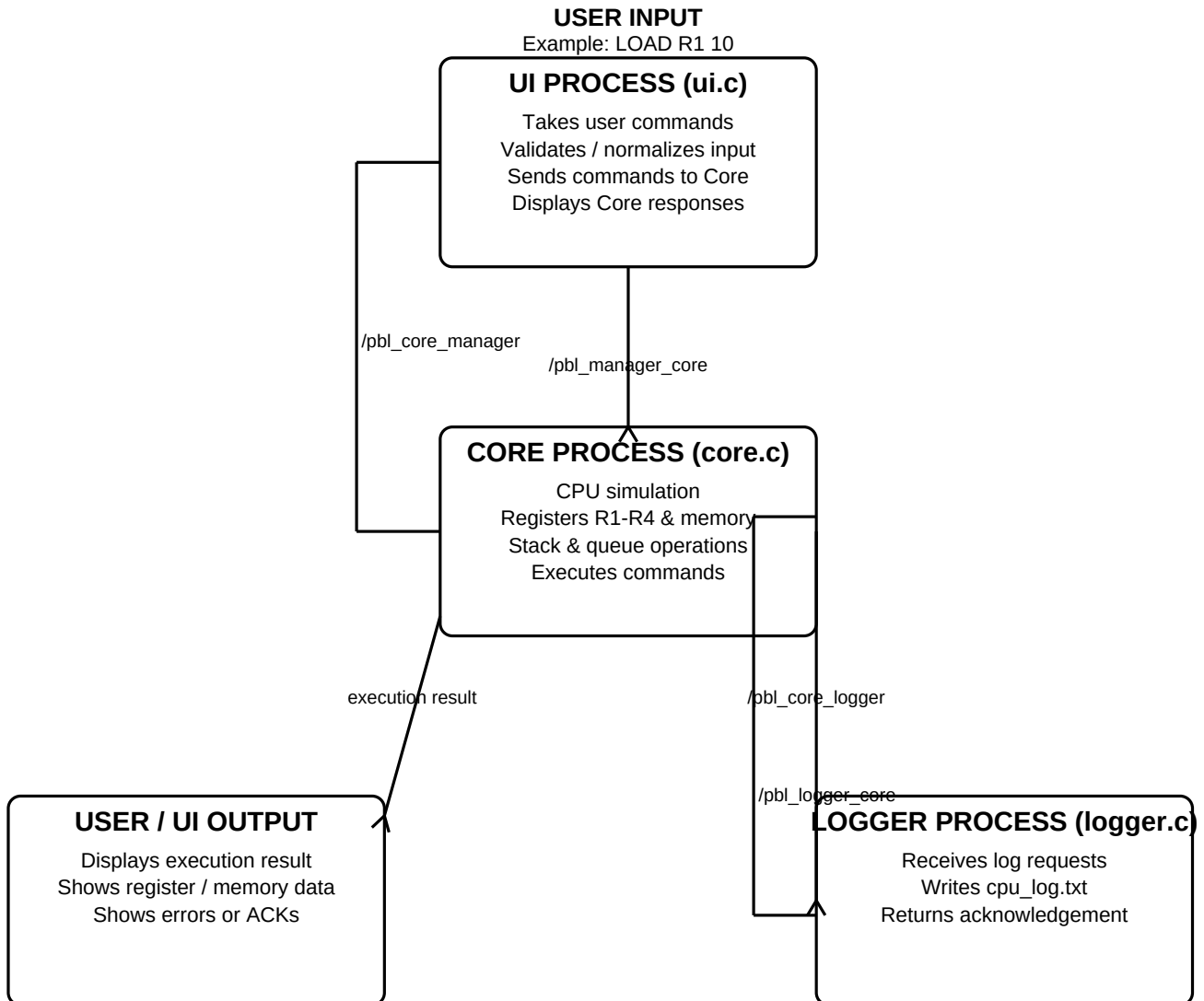


Multi-Process CPU Simulator

Working Architecture



Overall command flow:

USER → UI → Core → Logger → Core → UI → USER

IPC Queues and Process Responsibilities

Message Queue	Direction	Purpose
/pbl_manager_core	UI → Core	Sends user commands
/pbl_core_manager	Core → UI	Sends execution results
/pbl_core_logger	Core → Logger	Sends log requests
/pbl_logger_core	Logger → Core	Sends logger acknowledgement

Process responsibilities

UI Process (ui.c)

- Accepts commands from the user.
- Normalizes and sends commands to Core.
- Waits for and displays Core responses.

Core Process (core.c)

- Acts as the CPU simulator.
- Maintains registers, memory, stack and queue.
- Executes operations and communicates with UI and Logger.

Logger Process (logger.c)

- Receives execution log messages from Core.
- Writes execution information to cpu_log.txt.
- Returns an acknowledgement to Core.

Example: LOAD R1 10

1. User enters LOAD R1 10 in the UI.
2. UI sends the command through /pbl_manager_core.
3. Core receives and executes the LOAD operation.
4. Core sends a log request through /pbl_core_logger.
5. Logger records the event and acknowledges through /pbl_logger_core.
6. Core sends the execution result through /pbl_core_manager.
7. UI displays the result to the user.

Viva explanation:

“Our system has three independent processes—UI, Core and Logger—which communicate using POSIX Message Queues.”